



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ	Информатика и системы управления (ИУ)
КАФЕДРА	Информационная безопасность (ИУ8)
ДИСЦИПЛИНА	Математические основы искусственного интеллекта

ДОМАШНЕЕ ЗАДАНИЕ №1

Исследование экстремальных задач
название

Группа ИУ8-13М

Студент	5 вариант		Никулина З.Е.
		<i>подпись</i>	<i>фамилия, и.о.</i>

Преподаватель		Каганов Ю.Т.
	<i>подпись</i>	<i>фамилия, и.о.</i>

Москва, 2025 г.

Цель работы – ознакомление с основными понятиями экстремальных задач, исследование типов экстремумов для задач без ограничений и с ограничениями.

Практическая часть

1 Постановка задачи

Необходимо рассматривать функцию $f(x)$, $x = (x_1 \dots x_n)^T$, определённую в n -мерном евклидовом пространстве. Эта функция выступает в роли целевой, поскольку её значения характеризуют степень достижения поставленной цели, для которой и ставится задача.

В пространстве решений выделяется множество допустимых точек, среди которых проводится поиск. Каждая из этих точек соответствует определённому набору значений переменных, удовлетворяющему заданным условиям.

Требуется найти такой вектор x^e , который принадлежит множеству допустимых решений и при этом обеспечивает наименьшее значение целевой функции. Таким образом, задача сводится к определению точки, в которой достигается минимум функции при соблюдении всех заданных ограничений: $f(x^e) = \min_{x \in X} f(x)$.

2 Построение поверхности и линий уровня

Необходимо построить график поверхности и исследовать линии уровня для функции:

$$f(x_1, x_2) = (x_1 - 2x_2)^2 \text{ и } f(x_1, x_2) = (x_1 - 2x_2)^2 = \text{const} = r^2, \\ \text{где } r = 1, 2, 3, 4.$$

На рисунке 1 показана поверхность для заданной функции. Данная функция является параболоидом, который вытянут вдоль прямой $x_1 = 2x_2$.

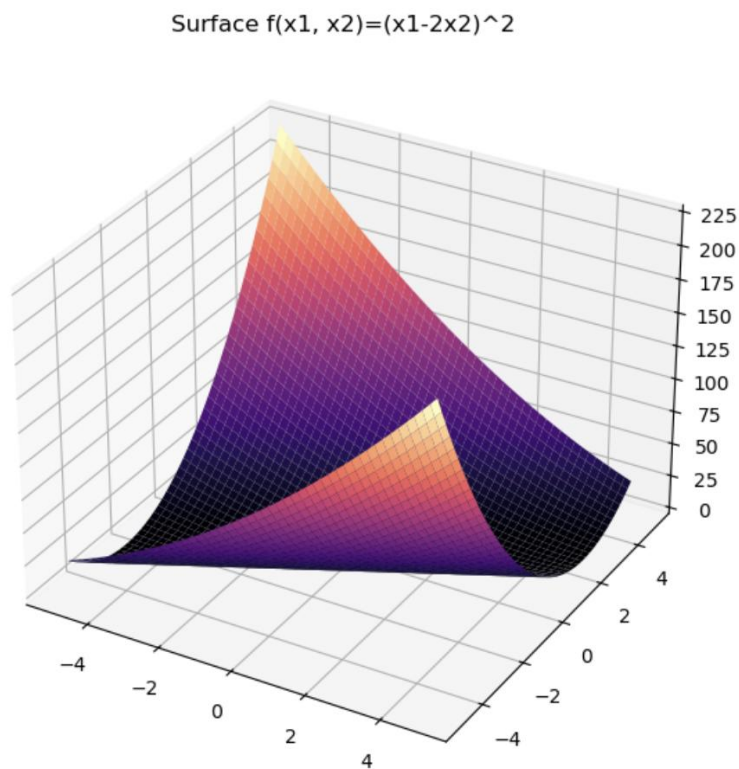


Рисунок 1 – График поверхности функции

На рисунке 2 приведен график для исследования линий уровня для различных значений r . Линии уровня представляют собой две параллельные прямые, смещенные относительно линии минимума $x_1 = 2x_2$ на расстояние, которое зависит от r .

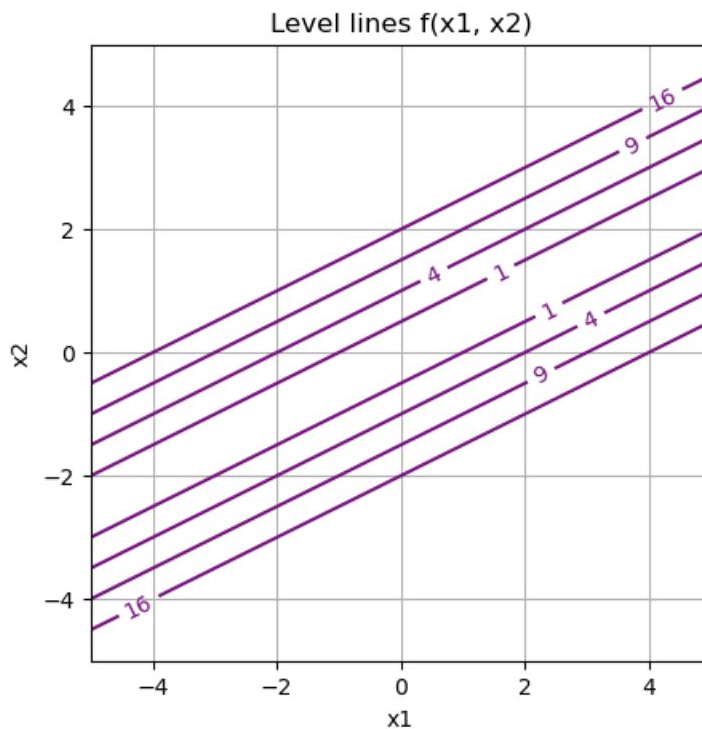


Рисунок 2 – Линии уровня для заданной функции

3 Исследование типов экстремумов для многоэкстремальных задач

Для функции Растригина в области поиска $x_i \in [-5, 5]$ и функции Стенгера в области поиска $x_i \in [-1, 4]$ были выявлены экстремумы типа минимума и максимума, а также определены локальные и глобальные экстремумы.

Функция Растригина имеет вид:

$$f(x_1, x_2) = \sum_{i=1}^2 (x_i^2 - c \cos(2\pi x_i) + p) + f^0,$$

где $c = 5, p = 10, f^0 = -100$ для варианта 5.

Функция Стенгера:

$$f(x_1, x_2) = (x_1^2 + ax_2)^2 + (x_2^2 + bx_1 + cx_2)^2,$$

где $a = -10, b = -1, c = +2$ для варианта 5.

На рисунке 3 показана функция Растригина и ее экстремумы, на рисунке 4 и 5 изображены проекция и поверхность функции соответственно.

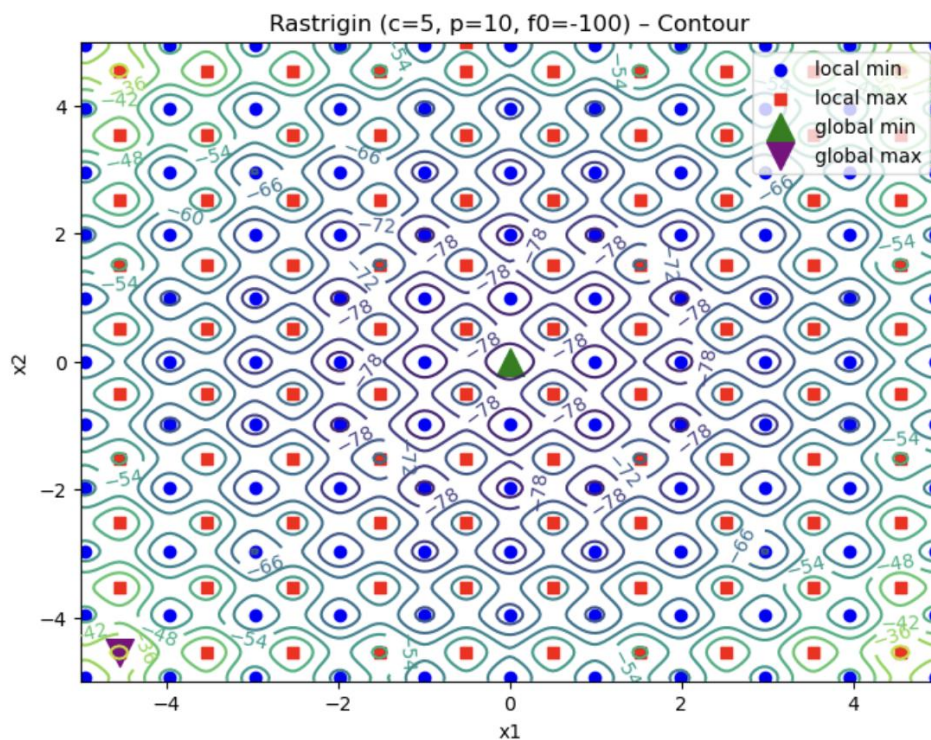


Рисунок 3 – Функция Растригина с линией уровня 10

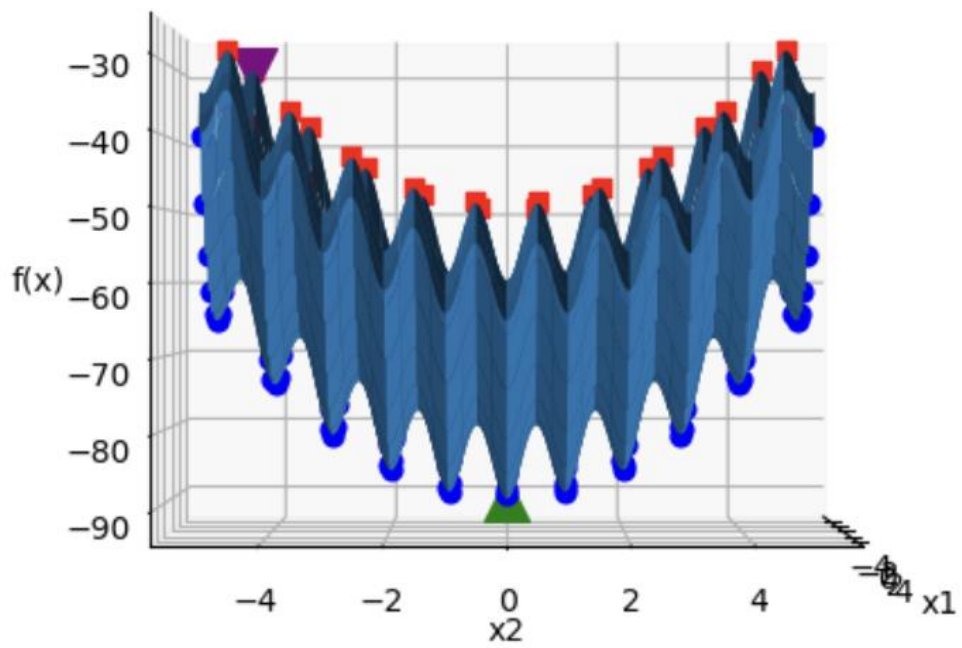


Рисунок 4 – Вид поверхности функции Растригина сбоку

Rastrigin (c=5, p=10, f0=-100) - Surface

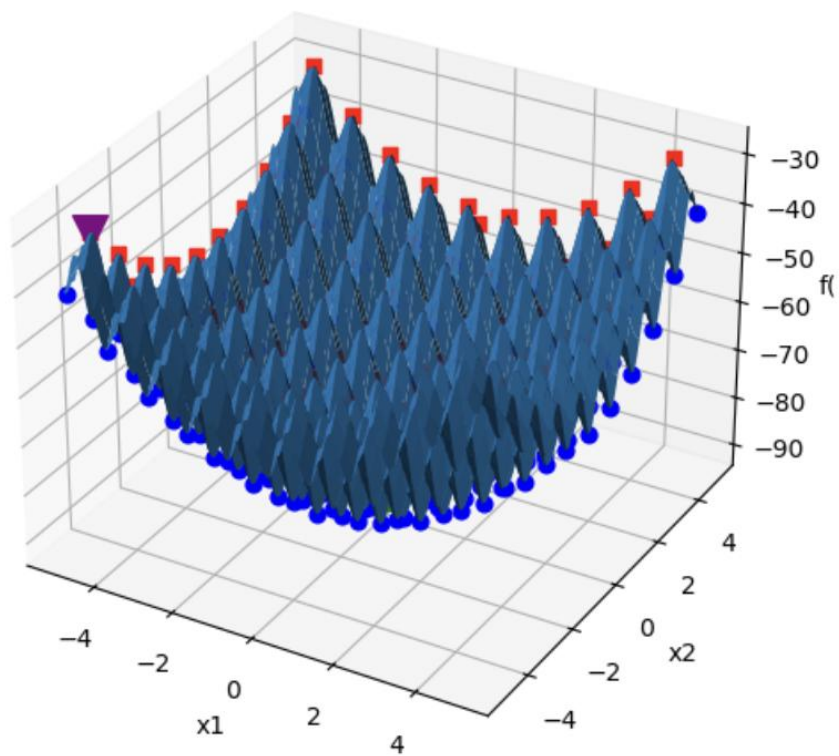


Рисунок 5 – Поверхность функции Растригина

Согласно разработанной программе были выявлены экстремумы типа минимум и максимум. На рисунке 6 выведены глобальные минимум и максимум. В Приложении А представлены локальные экстремумы для функции Растригина.

Global minimum:

$x^* = (0.000000, 0.000000)$, $f = -90.000000$, $feasible=True$

Global maximum:

$x^* = (-4.546737, -4.546737)$, $f = -29.082449$, $feasible=True$

Рисунок 6 – Результат работы программы

Глобальные экстремумы являются одновременно и локальными.

Функция Стенгера с линией уровня 10 и графические обозначения экстремумов показаны на рисунке 7. Проекция сбоку и сама поверхность функции изображены на рисунках 8 и 9.

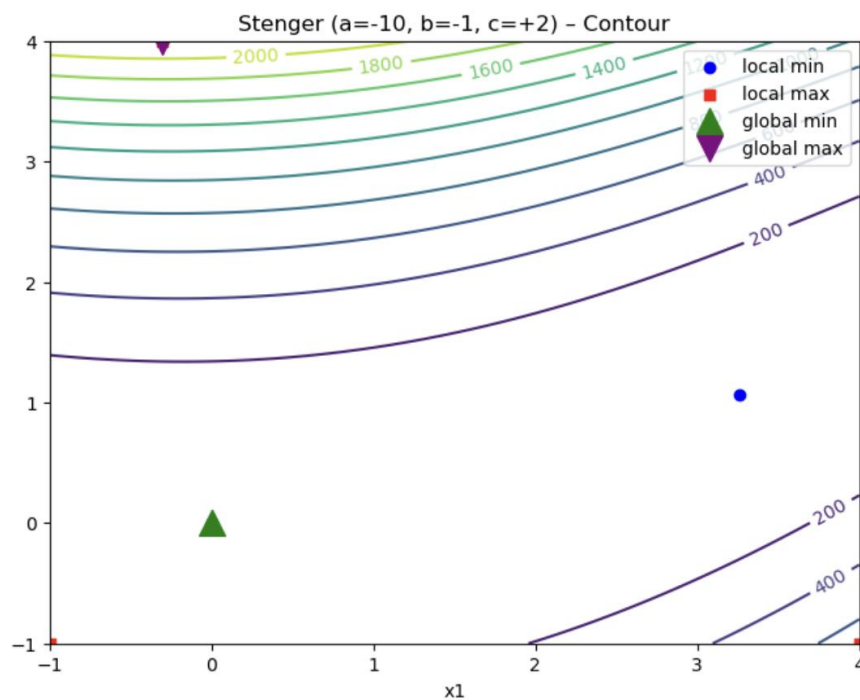


Рисунок 7 – Функция Стенгера с линией уровня 10

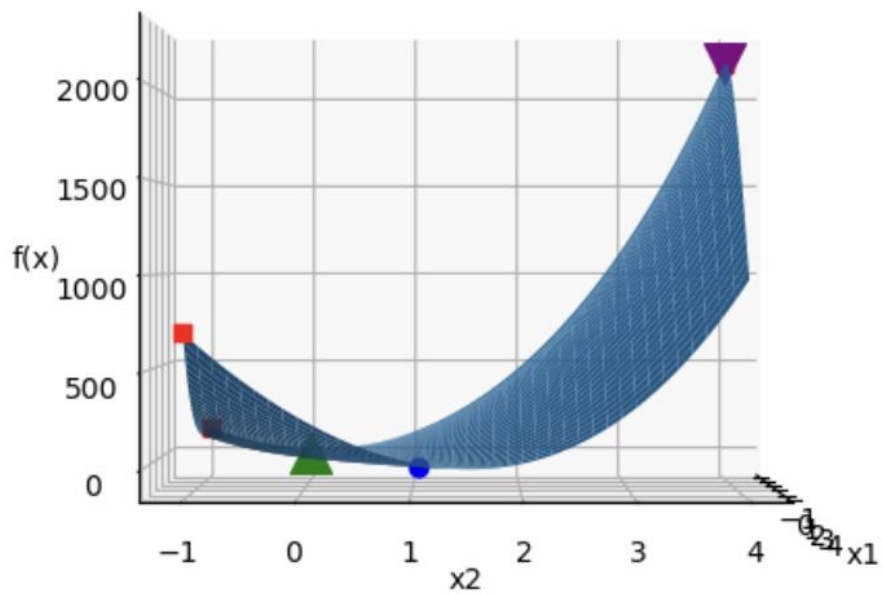


Рисунок 8 – Вид поверхности функции Стенгера сбоку

Stenger (a=-10, b=-1, c=+2) - Surface

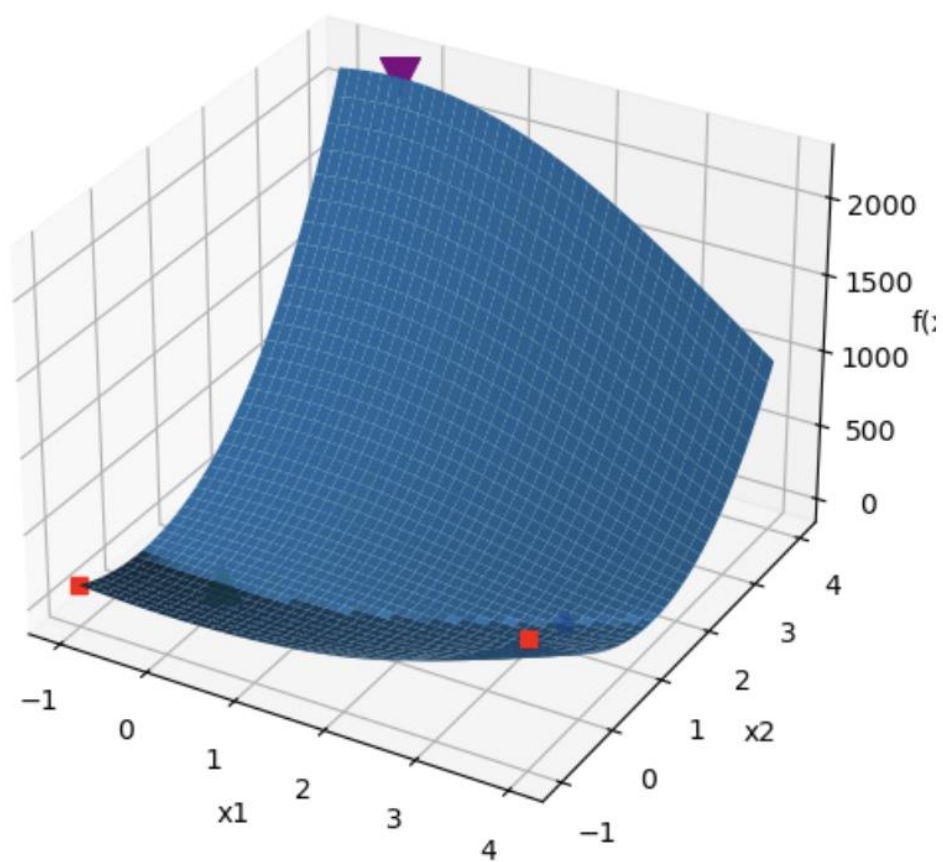


Рисунок 9 – Поверхность функции Стенгера

На рисунке 10 представлены результаты выявления глобальных минимума и максимума для функции Стенгера.

Global minimum:

$x^* = (0.000000, 0.000000), f = 0.000000, \text{feasible}=\text{True}$

Global maximum:

$x^* = (-0.304512, 4.000000), f = 2183.299697, \text{feasible}=\text{True}$

Рисунок 10 – Результаты работы программы

Локальные экстремумы для функции Стенгера показаны на рисунке 11.

Local min:

1. $x^* = (0.000000, 0.000000), f = 0.000000, \text{feasible}=\text{True}$

2. $x^* = (3.262967, 1.064695), f = 0.000000, \text{feasible}=\text{True}$

Local max:

1. $x^* = (-0.304512, 4.000000), f = 2183.299697, \text{feasible}=\text{True}$

2. $x^* = (4.000000, -1.000000), f = 701.000000, \text{feasible}=\text{True}$

3. $x^* = (-1.000000, -1.000000), f = 121.000000, \text{feasible}=\text{True}$

Рисунок 11 – Локальные экстремумы функции Стенгера

4 Исследование экстремальных задач с заданными ограничениями.

Для функции $f(x_1, x_2) = x_1^2 + x_2^2$ нужно было найти точки экстремума при ограничениях:

$$X = \{x | x_1 + x_2 \leq 1, x_1 \geq 0, x_2 \geq 0\}.$$

И для функции $f(x_1, x_2) = x_1^2 + x_2^2 - \frac{1}{2}x_1x_2$ – при ограничениях:

$$X = \{x | x_1^2 + 2x_2^2 \leq 1, x_1 \geq 0, x_2 \geq 0\}.$$

На рисунках 12-14 показаны условные экстремумы и их координаты согласно ограничениям 1 пункта.

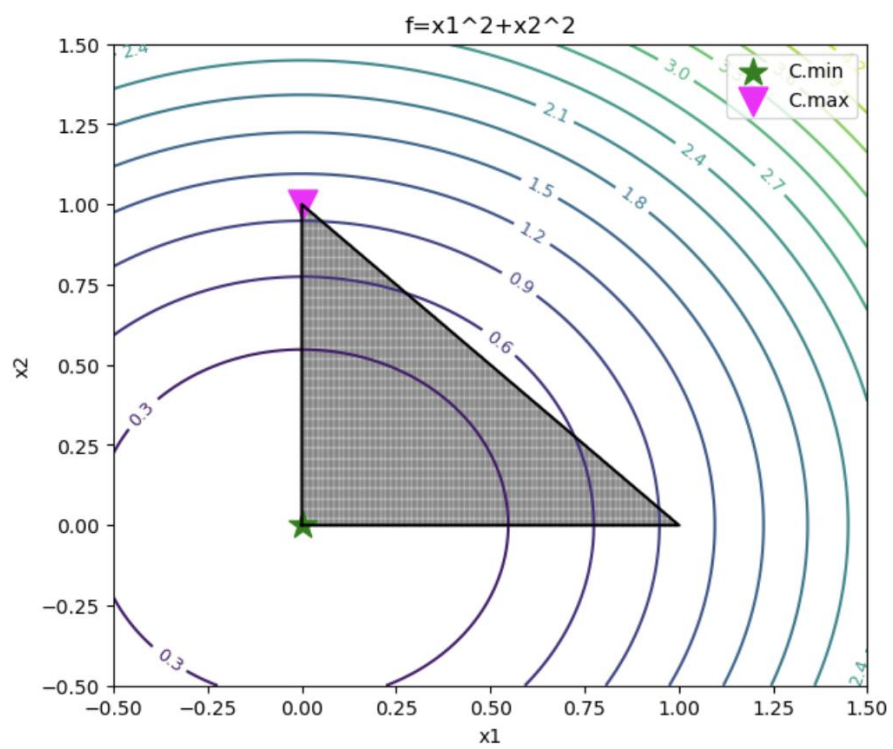


Рисунок 12 – Локальные экстремумы функции Стенгера

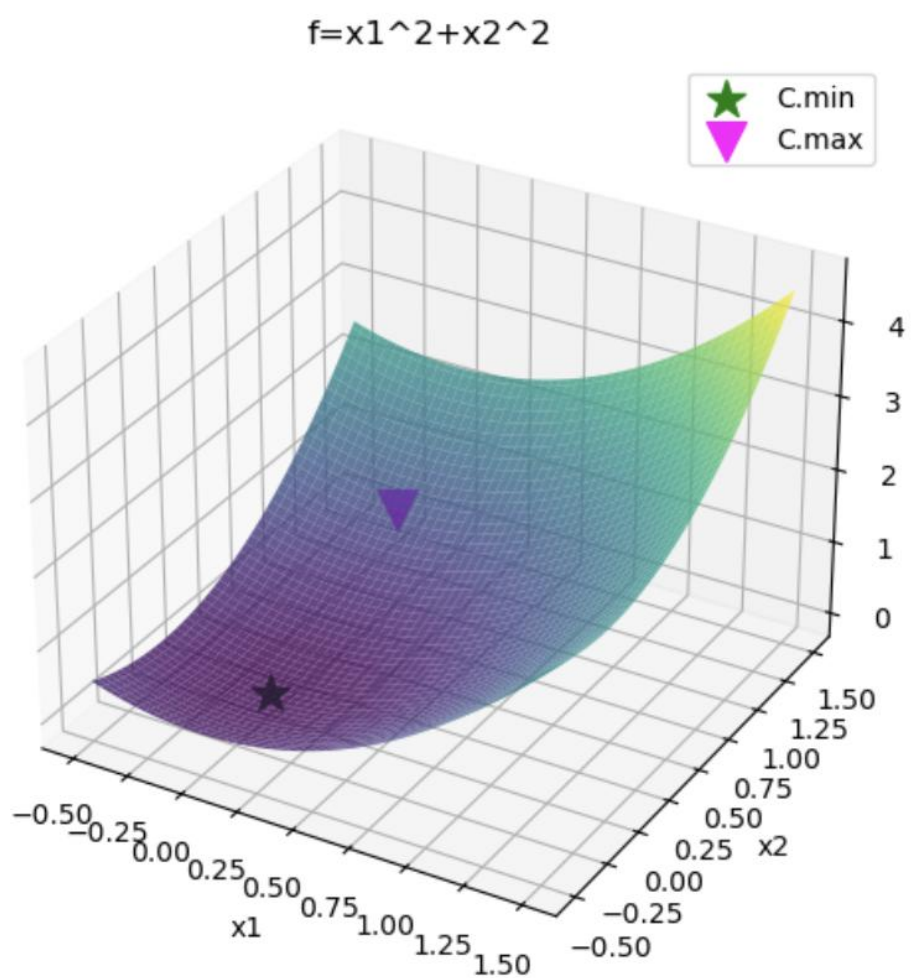


Рисунок 13 – Локальные экстремумы функции Стенгера

Условный минимум: $[0.00000000e+00 \ 1.13737334e-24]$ $f= 1.2936181196846227e-48$
 Условный максимум: $[4.16250368e-13 \ 1.00000000e+00]$ $f= 1.0000000000012617$

Рисунок 14 – Локальные экстремумы функции Стенгера

Условные экстремумы и их координаты для функции пункта 2 выведены на рисунках 15-17.

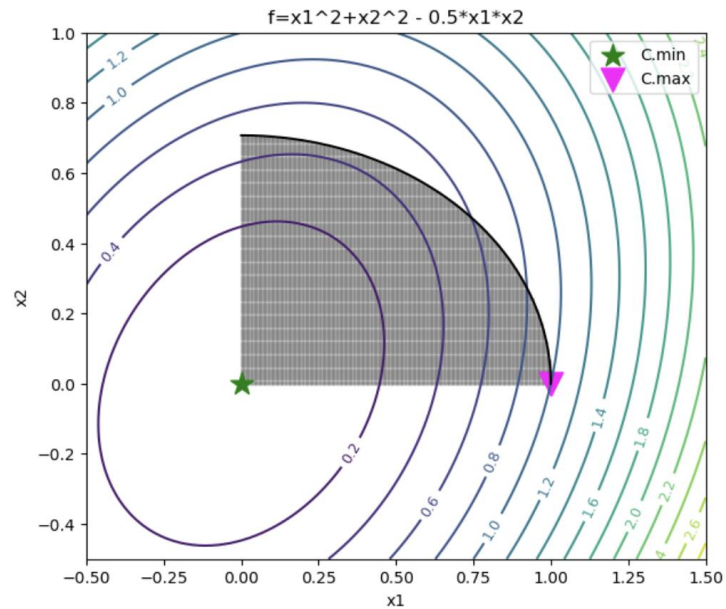


Рисунок 15 – Локальные экстремумы функции Стенгера

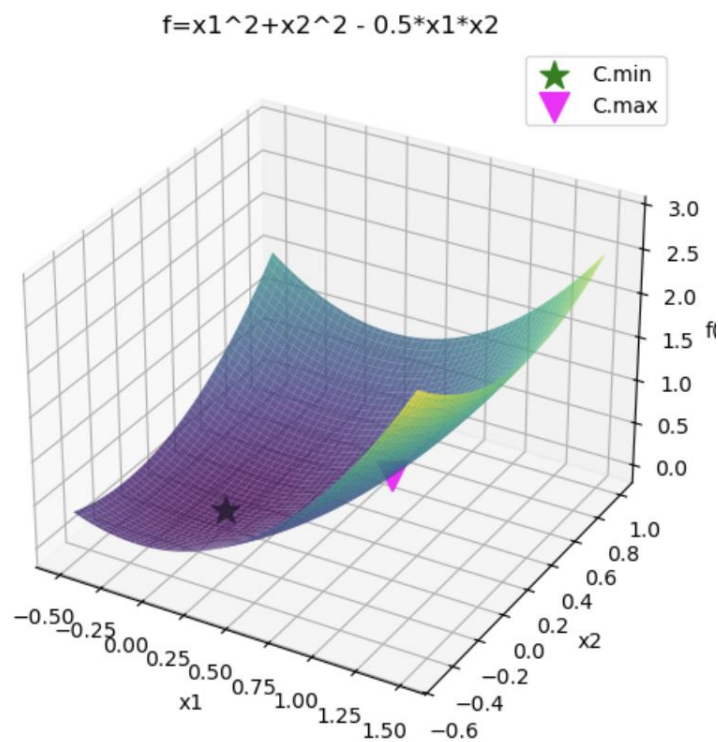


Рисунок 16 – Локальные экстремумы функции Стенгера

Условный минимум: [0. 0.] f= 0.0

Условный максимум: [1.00000000e+00 -3.29340992e-13] f= 1.0000000000767706

Рисунок 17 – Локальные экстремумы функции Стенгера

Листинги программ приведены в Приложении Б.

ПРИЛОЖЕНИЕ А

Локальные экстремумы для функции Растригина

Local min:

1. $x^* = (-4.948987, -4.948987)$, $f = -40.505759$, $\text{feasible}=\text{True}$
2. $x^* = (-3.959445, -4.948987)$, $f = -49.414225$, $\text{feasible}=\text{True}$
3. $x^* = (-2.969728, -4.948987)$, $f = -56.343423$, $\text{feasible}=\text{True}$
4. $x^* = (-1.979886, -4.948987)$, $f = -61.293055$, $\text{feasible}=\text{True}$
5. $x^* = (-0.989963, -4.948987)$, $f = -64.262914$, $\text{feasible}=\text{True}$
6. $x^* = (-0.000000, -4.948987)$, $f = -65.252880$, $\text{feasible}=\text{True}$
7. $x^* = (0.989963, -4.948987)$, $f = -64.262914$, $\text{feasible}=\text{True}$
8. $x^* = (1.979886, -4.948987)$, $f = -61.293055$, $\text{feasible}=\text{True}$
9. $x^* = (2.969728, -4.948987)$, $f = -56.343423$, $\text{feasible}=\text{True}$
10. $x^* = (3.959445, -4.948987)$, $f = -49.414225$, $\text{feasible}=\text{True}$
11. $x^* = (4.948987, -4.948987)$, $f = -40.505759$, $\text{feasible}=\text{True}$
12. $x^* = (-4.948987, -3.959445)$, $f = -49.414225$, $\text{feasible}=\text{True}$
13. $x^* = (-3.959445, -3.959445)$, $f = -58.322691$, $\text{feasible}=\text{True}$
14. $x^* = (-2.969728, -3.959445)$, $f = -65.251889$, $\text{feasible}=\text{True}$
15. $x^* = (-1.979886, -3.959445)$, $f = -70.201520$, $\text{feasible}=\text{True}$
16. $x^* = (-0.989963, -3.959445)$, $f = -73.171379$, $\text{feasible}=\text{True}$
17. $x^* = (-0.000000, -3.959445)$, $f = -74.161346$, $\text{feasible}=\text{True}$
18. $x^* = (0.989963, -3.959445)$, $f = -73.171379$, $\text{feasible}=\text{True}$
19. $x^* = (1.979886, -3.959445)$, $f = -70.201520$, $\text{feasible}=\text{True}$
20. $x^* = (2.969728, -3.959445)$, $f = -65.251889$, $\text{feasible}=\text{True}$
21. $x^* = (3.959445, -3.959445)$, $f = -58.322691$, $\text{feasible}=\text{True}$
22. $x^* = (4.948987, -3.959445)$, $f = -49.414225$, $\text{feasible}=\text{True}$
23. $x^* = (-4.948987, -2.969728)$, $f = -56.343423$, $\text{feasible}=\text{True}$
24. $x^* = (-3.959445, -2.969728)$, $f = -65.251889$, $\text{feasible}=\text{True}$
25. $x^* = (-2.969728, -2.969728)$, $f = -72.181087$, $\text{feasible}=\text{True}$
26. $x^* = (-1.979886, -2.969728)$, $f = -77.130718$, $\text{feasible}=\text{True}$
27. $x^* = (-0.989963, -2.969728)$, $f = -80.100577$, $\text{feasible}=\text{True}$
28. $x^* = (0.000000, -2.969728)$, $f = -81.090543$, $\text{feasible}=\text{True}$
29. $x^* = (0.989963, -2.969728)$, $f = -80.100577$, $\text{feasible}=\text{True}$
30. $x^* = (1.979886, -2.969728)$, $f = -77.130718$, $\text{feasible}=\text{True}$
31. $x^* = (2.969728, -2.969728)$, $f = -72.181087$, $\text{feasible}=\text{True}$
32. $x^* = (3.959445, -2.969728)$, $f = -65.251889$, $\text{feasible}=\text{True}$
33. $x^* = (4.948987, -2.969728)$, $f = -56.343423$, $\text{feasible}=\text{True}$
34. $x^* = (-4.948987, -1.979886)$, $f = -61.293055$, $\text{feasible}=\text{True}$
35. $x^* = (-3.959445, -1.979886)$, $f = -70.201520$, $\text{feasible}=\text{True}$
36. $x^* = (-2.969728, -1.979886)$, $f = -77.130718$, $\text{feasible}=\text{True}$
37. $x^* = (-1.979886, -1.979886)$, $f = -82.080350$, $\text{feasible}=\text{True}$
38. $x^* = (-0.989963, -1.979886)$, $f = -85.050209$, $\text{feasible}=\text{True}$
39. $x^* = (-0.000000, -1.979886)$, $f = -86.040175$, $\text{feasible}=\text{True}$
40. $x^* = (0.989963, -1.979886)$, $f = -85.050209$, $\text{feasible}=\text{True}$
41. $x^* = (1.979886, -1.979886)$, $f = -82.080350$, $\text{feasible}=\text{True}$
42. $x^* = (2.969728, -1.979886)$, $f = -77.130718$, $\text{feasible}=\text{True}$
43. $x^* = (3.959445, -1.979886)$, $f = -70.201520$, $\text{feasible}=\text{True}$
44. $x^* = (4.948987, -1.979886)$, $f = -61.293055$, $\text{feasible}=\text{True}$
45. $x^* = (-4.948987, -0.989963)$, $f = -64.262914$, $\text{feasible}=\text{True}$
46. $x^* = (-3.959445, -0.989963)$, $f = -73.171379$, $\text{feasible}=\text{True}$
47. $x^* = (-2.969728, -0.989963)$, $f = -80.100577$, $\text{feasible}=\text{True}$
48. $x^* = (-1.979886, -0.989963)$, $f = -85.050209$, $\text{feasible}=\text{True}$
49. $x^* = (-0.989963, -0.989963)$, $f = -88.020068$, $\text{feasible}=\text{True}$
50. $x^* = (0.000000, -0.989963)$, $f = -89.010034$, $\text{feasible}=\text{True}$
51. $x^* = (0.989963, -0.989963)$, $f = -88.020068$, $\text{feasible}=\text{True}$
52. $x^* = (1.979886, -0.989963)$, $f = -85.050209$, $\text{feasible}=\text{True}$
53. $x^* = (2.969728, -0.989963)$, $f = -80.100577$, $\text{feasible}=\text{True}$
54. $x^* = (3.959445, -0.989963)$, $f = -73.171379$, $\text{feasible}=\text{True}$
55. $x^* = (4.948987, -0.989963)$, $f = -64.262914$, $\text{feasible}=\text{True}$

56. $x^* = (-4.948987, -0.000000)$, $f = -65.252880$, feasible=True
 57. $x^* = (-3.959445, -0.000000)$, $f = -74.161346$, feasible=True
 58. $x^* = (-2.969728, 0.000000)$, $f = -81.090543$, feasible=True
 59. $x^* = (-1.979886, -0.000000)$, $f = -86.040175$, feasible=True
 60. $x^* = (-0.989963, 0.000000)$, $f = -89.010034$, feasible=True
 61. $x^* = (0.000000, 0.000000)$, $f = -90.000000$, feasible=True
 62. $x^* = (0.989963, 0.000000)$, $f = -89.010034$, feasible=True
 63. $x^* = (1.979886, -0.000000)$, $f = -86.040175$, feasible=True
 64. $x^* = (2.969728, 0.000000)$, $f = -81.090543$, feasible=True
 65. $x^* = (3.959445, -0.000000)$, $f = -74.161346$, feasible=True
 66. $x^* = (4.948987, -0.000000)$, $f = -65.252880$, feasible=True
 67. $x^* = (-4.948987, 0.989963)$, $f = -64.262914$, feasible=True
 68. $x^* = (-3.959445, 0.989963)$, $f = -73.171379$, feasible=True
 69. $x^* = (-2.969728, 0.989963)$, $f = -80.100577$, feasible=True
 70. $x^* = (-1.979886, 0.989963)$, $f = -85.050209$, feasible=True
 71. $x^* = (-0.989963, 0.989963)$, $f = -88.020068$, feasible=True
 72. $x^* = (0.000000, 0.989963)$, $f = -89.010034$, feasible=True
 73. $x^* = (0.989963, 0.989963)$, $f = -88.020068$, feasible=True
 74. $x^* = (1.979886, 0.989963)$, $f = -85.050209$, feasible=True
 75. $x^* = (2.969728, 0.989963)$, $f = -80.100577$, feasible=True
 76. $x^* = (3.959445, 0.989963)$, $f = -73.171379$, feasible=True
 77. $x^* = (4.948987, 0.989963)$, $f = -64.262914$, feasible=True
 78. $x^* = (-4.948987, 1.979886)$, $f = -61.293055$, feasible=True
 79. $x^* = (-3.959445, 1.979886)$, $f = -70.201520$, feasible=True
 80. $x^* = (-2.969728, 1.979886)$, $f = -77.130718$, feasible=True
 81. $x^* = (-1.979886, 1.979886)$, $f = -82.080350$, feasible=True
 82. $x^* = (-0.989963, 1.979886)$, $f = -85.050209$, feasible=True
 83. $x^* = (-0.000000, 1.979886)$, $f = -86.040175$, feasible=True
 84. $x^* = (0.989963, 1.979886)$, $f = -85.050209$, feasible=True
 85. $x^* = (1.979886, 1.979886)$, $f = -82.080350$, feasible=True
 86. $x^* = (2.969728, 1.979886)$, $f = -77.130718$, feasible=True
 87. $x^* = (3.959445, 1.979886)$, $f = -70.201520$, feasible=True
 88. $x^* = (4.948987, 1.979886)$, $f = -61.293055$, feasible=True
 89. $x^* = (-4.948987, 2.969728)$, $f = -56.343423$, feasible=True
 90. $x^* = (-3.959445, 2.969728)$, $f = -65.251889$, feasible=True
 91. $x^* = (-2.969728, 2.969728)$, $f = -72.181087$, feasible=True
 92. $x^* = (-1.979886, 2.969728)$, $f = -77.130718$, feasible=True
 93. $x^* = (-0.989963, 2.969728)$, $f = -80.100577$, feasible=True
 94. $x^* = (0.000000, 2.969728)$, $f = -81.090543$, feasible=True
 95. $x^* = (0.989963, 2.969728)$, $f = -80.100577$, feasible=True
 96. $x^* = (1.979886, 2.969728)$, $f = -77.130718$, feasible=True
 97. $x^* = (2.969728, 2.969728)$, $f = -72.181087$, feasible=True
 98. $x^* = (3.959445, 2.969728)$, $f = -65.251889$, feasible=True
 99. $x^* = (4.948987, 2.969728)$, $f = -56.343423$, feasible=True
 100. $x^* = (-4.948987, 3.959445)$, $f = -49.414225$, feasible=True
 101. $x^* = (-3.959445, 3.959445)$, $f = -58.322691$, feasible=True
 102. $x^* = (-2.969728, 3.959445)$, $f = -65.251889$, feasible=True
 103. $x^* = (-1.979886, 3.959445)$, $f = -70.201520$, feasible=True
 104. $x^* = (-0.989963, 3.959445)$, $f = -73.171379$, feasible=True
 105. $x^* = (-0.000000, 3.959445)$, $f = -74.161346$, feasible=True
 106. $x^* = (0.989963, 3.959445)$, $f = -73.171379$, feasible=True
 107. $x^* = (1.979886, 3.959445)$, $f = -70.201520$, feasible=True
 108. $x^* = (2.969728, 3.959445)$, $f = -65.251889$, feasible=True
 109. $x^* = (3.959445, 3.959445)$, $f = -58.322691$, feasible=True
 110. $x^* = (4.948987, 3.959445)$, $f = -49.414225$, feasible=True
 111. $x^* = (-4.948987, 4.948987)$, $f = -40.505759$, feasible=True
 112. $x^* = (-3.959445, 4.948987)$, $f = -49.414225$, feasible=True
 113. $x^* = (-2.969728, 4.948987)$, $f = -56.343423$, feasible=True
 114. $x^* = (-1.979886, 4.948987)$, $f = -61.293055$, feasible=True
 115. $x^* = (-0.989963, 4.948987)$, $f = -64.262914$, feasible=True
 116. $x^* = (-0.000000, 4.948987)$, $f = -65.252880$, feasible=True
 117. $x^* = (0.989963, 4.948987)$, $f = -64.262914$, feasible=True

118. $x^* = (1.979886, 4.948987)$, $f = -61.293055$, feasible=True
 119. $x^* = (2.969728, 4.948987)$, $f = -56.343423$, feasible=True
 120. $x^* = (3.959445, 4.948987)$, $f = -49.414225$, feasible=True
 121. $x^* = (4.948987, 4.948987)$, $f = -40.505759$, feasible=True

Local max:

1. $x^* = (-4.546737, -4.546737)$, $f = -29.082449$, feasible=True
 2. $x^* = (-3.536138, -4.546737)$, $f = -37.165292$, feasible=True
 3. $x^* = (-2.525702, -4.546737)$, $f = -43.227110$, feasible=True
 4. $x^* = (-1.515378, -4.546737)$, $f = -47.268176$, feasible=True
 5. $x^* = (-0.505119, -4.546737)$, $f = -49.288665$, feasible=True
 6. $x^* = (0.505119, -4.546737)$, $f = -49.288665$, feasible=True
 7. $x^* = (1.515378, -4.546737)$, $f = -47.268176$, feasible=True
 8. $x^* = (2.525702, -4.546737)$, $f = -43.227110$, feasible=True
 9. $x^* = (3.536138, -4.546737)$, $f = -37.165292$, feasible=True
 10. $x^* = (4.546737, -4.546737)$, $f = -29.082449$, feasible=True
 11. $x^* = (-4.546737, -3.536138)$, $f = -37.165292$, feasible=True
 12. $x^* = (-3.536138, -3.536138)$, $f = -45.248136$, feasible=True
 13. $x^* = (-2.525702, -3.536138)$, $f = -51.309954$, feasible=True
 14. $x^* = (-1.515378, -3.536138)$, $f = -55.351019$, feasible=True
 15. $x^* = (-0.505119, -3.536138)$, $f = -57.371509$, feasible=True
 16. $x^* = (0.505119, -3.536138)$, $f = -57.371509$, feasible=True
 17. $x^* = (1.515378, -3.536138)$, $f = -55.351019$, feasible=True
 18. $x^* = (2.525702, -3.536138)$, $f = -51.309954$, feasible=True
 19. $x^* = (3.536138, -3.536138)$, $f = -45.248136$, feasible=True
 20. $x^* = (4.546737, -3.536138)$, $f = -37.165292$, feasible=True
 21. $x^* = (-4.546737, -2.525702)$, $f = -43.227110$, feasible=True
 22. $x^* = (-3.536138, -2.525702)$, $f = -51.309954$, feasible=True
 23. $x^* = (-2.525702, -2.525702)$, $f = -57.371771$, feasible=True
 24. $x^* = (-1.515378, -2.525702)$, $f = -61.412837$, feasible=True
 25. $x^* = (-0.505119, -2.525702)$, $f = -63.433327$, feasible=True
 26. $x^* = (0.505119, -2.525702)$, $f = -63.433327$, feasible=True
 27. $x^* = (1.515378, -2.525702)$, $f = -61.412837$, feasible=True
 28. $x^* = (2.525702, -2.525702)$, $f = -57.371771$, feasible=True
 29. $x^* = (3.536138, -2.525702)$, $f = -51.309954$, feasible=True
 30. $x^* = (4.546737, -2.525702)$, $f = -43.227110$, feasible=True
 31. $x^* = (-4.546737, -1.515378)$, $f = -47.268176$, feasible=True
 32. $x^* = (-3.536138, -1.515378)$, $f = -55.351019$, feasible=True
 33. $x^* = (-2.525702, -1.515378)$, $f = -61.412837$, feasible=True
 34. $x^* = (-1.515378, -1.515378)$, $f = -65.453903$, feasible=True
 35. $x^* = (-0.505119, -1.515378)$, $f = -67.474392$, feasible=True
 36. $x^* = (0.505119, -1.515378)$, $f = -67.474392$, feasible=True
 37. $x^* = (1.515378, -1.515378)$, $f = -65.453903$, feasible=True
 38. $x^* = (2.525702, -1.515378)$, $f = -61.412837$, feasible=True
 39. $x^* = (3.536138, -1.515378)$, $f = -55.351019$, feasible=True
 40. $x^* = (4.546737, -1.515378)$, $f = -47.268176$, feasible=True
 41. $x^* = (-4.546737, -0.505119)$, $f = -49.288665$, feasible=True
 42. $x^* = (-3.536138, -0.505119)$, $f = -57.371509$, feasible=True
 43. $x^* = (-2.525702, -0.505119)$, $f = -63.433327$, feasible=True
 44. $x^* = (-1.515378, -0.505119)$, $f = -67.474392$, feasible=True
 45. $x^* = (-0.505119, -0.505119)$, $f = -69.494882$, feasible=True
 46. $x^* = (0.505119, -0.505119)$, $f = -69.494882$, feasible=True
 47. $x^* = (1.515378, -0.505119)$, $f = -67.474392$, feasible=True
 48. $x^* = (2.525702, -0.505119)$, $f = -63.433327$, feasible=True
 49. $x^* = (3.536138, -0.505119)$, $f = -57.371509$, feasible=True
 50. $x^* = (4.546737, -0.505119)$, $f = -49.288665$, feasible=True
 51. $x^* = (-4.546737, 0.505119)$, $f = -49.288665$, feasible=True
 52. $x^* = (-3.536138, 0.505119)$, $f = -57.371509$, feasible=True
 53. $x^* = (-2.525702, 0.505119)$, $f = -63.433327$, feasible=True
 54. $x^* = (-1.515378, 0.505119)$, $f = -67.474392$, feasible=True
 55. $x^* = (-0.505119, 0.505119)$, $f = -69.494882$, feasible=True
 56. $x^* = (0.505119, 0.505119)$, $f = -69.494882$, feasible=True

57. $x^* = (1.515378, 0.505119)$, $f = -67.474392$, feasible=True
 58. $x^* = (2.525702, 0.505119)$, $f = -63.433327$, feasible=True
 59. $x^* = (3.536138, 0.505119)$, $f = -57.371509$, feasible=True
 60. $x^* = (4.546737, 0.505119)$, $f = -49.288665$, feasible=True
 61. $x^* = (-4.546737, 1.515378)$, $f = -47.268176$, feasible=True
 62. $x^* = (-3.536138, 1.515378)$, $f = -55.351019$, feasible=True
 63. $x^* = (-2.525702, 1.515378)$, $f = -61.412837$, feasible=True
 64. $x^* = (-1.515378, 1.515378)$, $f = -65.453903$, feasible=True
 65. $x^* = (-0.505119, 1.515378)$, $f = -67.474392$, feasible=True
 66. $x^* = (0.505119, 1.515378)$, $f = -67.474392$, feasible=True
 67. $x^* = (1.515378, 1.515378)$, $f = -65.453903$, feasible=True
 68. $x^* = (2.525702, 1.515378)$, $f = -61.412837$, feasible=True
 69. $x^* = (3.536138, 1.515378)$, $f = -55.351019$, feasible=True
 70. $x^* = (4.546737, 1.515378)$, $f = -47.268176$, feasible=True
 71. $x^* = (-4.546737, 2.525702)$, $f = -43.227110$, feasible=True
 72. $x^* = (-3.536138, 2.525702)$, $f = -51.309954$, feasible=True
 73. $x^* = (-2.525702, 2.525702)$, $f = -57.371771$, feasible=True
 74. $x^* = (-1.515378, 2.525702)$, $f = -61.412837$, feasible=True
 75. $x^* = (-0.505119, 2.525702)$, $f = -63.433327$, feasible=True
 76. $x^* = (0.505119, 2.525702)$, $f = -63.433327$, feasible=True
 77. $x^* = (1.515378, 2.525702)$, $f = -61.412837$, feasible=True
 78. $x^* = (2.525702, 2.525702)$, $f = -57.371771$, feasible=True
 79. $x^* = (3.536138, 2.525702)$, $f = -51.309954$, feasible=True
 80. $x^* = (4.546737, 2.525702)$, $f = -43.227110$, feasible=True
 81. $x^* = (-4.546737, 3.536138)$, $f = -37.165292$, feasible=True
 82. $x^* = (-3.536138, 3.536138)$, $f = -45.248136$, feasible=True
 83. $x^* = (-2.525702, 3.536138)$, $f = -51.309954$, feasible=True
 84. $x^* = (-1.515378, 3.536138)$, $f = -55.351019$, feasible=True
 85. $x^* = (-0.505119, 3.536138)$, $f = -57.371509$, feasible=True
 86. $x^* = (0.505119, 3.536138)$, $f = -57.371509$, feasible=True
 87. $x^* = (1.515378, 3.536138)$, $f = -55.351019$, feasible=True
 88. $x^* = (2.525702, 3.536138)$, $f = -51.309954$, feasible=True
 89. $x^* = (3.536138, 3.536138)$, $f = -45.248136$, feasible=True
 90. $x^* = (4.546737, 3.536138)$, $f = -37.165292$, feasible=True
 91. $x^* = (-4.546737, 4.546737)$, $f = -29.082449$, feasible=True
 92. $x^* = (-3.536138, 4.546737)$, $f = -37.165292$, feasible=True
 93. $x^* = (-2.525702, 4.546737)$, $f = -43.227110$, feasible=True
 94. $x^* = (-1.515378, 4.546737)$, $f = -47.268176$, feasible=True
 95. $x^* = (-0.505119, 4.546737)$, $f = -49.288665$, feasible=True
 96. $x^* = (0.505119, 4.546737)$, $f = -49.288665$, feasible=True
 97. $x^* = (1.515378, 4.546737)$, $f = -47.268176$, feasible=True
 98. $x^* = (2.525702, 4.546737)$, $f = -43.227110$, feasible=True
 99. $x^* = (3.536138, 4.546737)$, $f = -37.165292$, feasible=True
 100. $x^* = (4.546737, 4.546737)$, $f = -29.082449$, feasible=True
 101. $x^* = (-0.505119, 5.000000)$, $f = -54.747441$, feasible=True

ПРИЛОЖЕНИЕ Б

Листинги программ

```
#Задание 1
x1 = np.linspace(-5, 5, 400)
x2 = np.linspace(-5, 5, 400)
X1, X2 = np.meshgrid(x1, x2)
F = (X1 - 2*X2)**2

fig = plt.figure(figsize=(7,7))
ax = fig.add_subplot(projection='3d')
ax.plot_surface(X1, X2, F, cmap='magma')
ax.set_title("Surface  $f(x_1, x_2)=(x_1-2x_2)^2$ ")
plt.show()

plt.figure(figsize=(5,5))
rv = [1,2,3,4]
levels = [r**2 for r in rv]
contours = plt.contour(X1, X2, F, levels=levels, colors='purple')
plt.clabel(contours)
plt.title("Level lines  $f(x_1, x_2)$ ")
plt.xlabel("x1")
plt.ylabel("x2")
plt.grid(True)
plt.show()

#Задание 2
def rastrigin(x, c=5.0, p=10.0, f0=-100.0):
    x = np.asarray(x)
    return np.sum(x**2 - c*np.cos(2*np.pi*x) + p) + f0

def stenger(x, a=-10.0, b=-1.0, c=+2.0):
    x1, x2 = x
    return (x1**2 + a*x2)**2 + (x2**2 + b*x1 + c*x2)**2

dom_r = [-5.0, 5.0]
dom_s = [-1.0, 4.0]
def in_bounds(x, bounds):
    return (bounds[0][0] <= x[0] <= bounds[0][1]) and (bounds[1][0] <= x[1] <=
bounds[1][1])
def classify_local_extremum(f, x, bounds, eps=1e-2, grid_pts=51):
    x = np.asarray(x, dtype=float)
    dx = eps
    xs = np.linspace(max(bounds[0][0], x[0]-dx), min(bounds[0][1], x[0]+dx), grid_pts)
    ys = np.linspace(max(bounds[1][0], x[1]-dx), min(bounds[1][1], x[1]+dx), grid_pts)
    XX, YY = np.meshgrid(xs, ys, indexing="xy")
    Z = np.array([[f([xx, yy]) for xx, yy in zip(row_x, row_y)] for row_x, row_y in
zip(XX, YY)])
    f0 = f(x)
    is_min = np.all(f0 <= Z + 1e-10)
    is_max = np.all(f0 >= Z - 1e-10)
    if is_min and not is_max:
        return "local_min"
    if is_max and not is_min:
        return "local_max"
    if is_min and is_max:
        return "flat"
    return "nn"
def unique(points, tol=1e-3):
    if points is None or len(points)==0:
```

```

        return np.empty((0,2))
    uniq = []
    for p in np.asarray(points):
        if not any(np.linalg.norm(p - q) < tol for q in uniq):
            uniq.append(p)
    return np.array(uniq) if len(uniq)>0 else np.empty((0,2))
def search_extrema(f, bounds, n_grid=201, n_starts=100, refine=True):
    xs = np.linspace(bounds[0][0], bounds[0][1], n_grid)
    ys = np.linspace(bounds[1][0], bounds[1][1], n_grid)
    XX, YY = np.meshgrid(xs, ys, indexing="xy")
    Z = np.empty_like(XX)
    for i in range(XX.shape[0]):
        for j in range(XX.shape[1]):
            Z[i, j] = f([XX[i, j], YY[i, j]])
    def neighborhood_extrema(is_min=True):
        cand = []
        for i in range(1, Z.shape[0]-1):
            for j in range(1, Z.shape[1]-1):
                center = Z[i, j]
                neigh = Z[i-1:i+2, j-1:j+2]
                if is_min:
                    if np.all(center <= neigh) and np.any(center < neigh):
                        cand.append(np.array([XX[i, j], YY[i, j]]))
                else:
                    if np.all(center >= neigh) and np.any(center > neigh):
                        cand.append(np.array([XX[i, j], YY[i, j]]))
        return unique(np.array(cand) if cand else np.empty((0,2)))
    min_cands = neighborhood_extrema(True)
    max_cands = neighborhood_extrema(False)

    rng = np.random.default_rng(42)
    random_pts = rng.uniform([bounds[0][0], bounds[1][0]], [bounds[0][1],
bounds[1][1]], size=(n_starts, 2))
    min_cands = np.vstack([min_cands, random_pts[: n_starts//2]]) if min_cands.size
else random_pts[: n_starts//2]
    max_cands = np.vstack([max_cands, random_pts[n_starts//2 :]]) if max_cands.size
else random_pts[n_starts//2 :]

    def project(x):
        x = np.asarray(x, float).copy()
        x[0] = np.clip(x[0], bounds[0][0], bounds[0][1])
        x[1] = np.clip(x[1], bounds[1][0], bounds[1][1])
        return x

    refined_mins = []
    refined_maxs = []
    if refine and SCIPY_OK:
        for x0 in min_cands:
            res = minimize(lambda v: f(project(v)), x0, method="Nelder-Mead", op-
tions=dict(maxiter=300, xatol=1e-8, fatol=1e-8))
            refined_mins.append(project(res.x))
        for x0 in max_cands:
            res = minimize(lambda v: -f(project(v)), x0, method="Nelder-Mead", op-
tions=dict(maxiter=300, xatol=1e-8, fatol=1e-8))
            refined_maxs.append(project(res.x))
        refined_mins = unique(np.array(refined_mins))
        refined_maxs = unique(np.array(refined_maxs))
    else:
        refined_mins = unique(min_cands)
        refined_maxs = unique(max_cands)

    bounds_pair = [tuple(bounds[0]), tuple(bounds[1])]

```

```

local_mins = []
local_maxs = []
for x in refined_mins:
    cl = classify_local_extremum(f, x, bounds_pair, eps=1e-2, grid_pts=11)
    if cl == "local_min":
        local_mins.append((x, f(x)))
for x in refined_maxs:
    cl = classify_local_extremum(f, x, bounds_pair, eps=1e-2, grid_pts=11)
    if cl == "local_max":
        local_maxs.append((x, f(x)))

if local_mins:
    gm = min(local_mins, key=lambda t: t[1])
else:
    flat_idx = np.argmin(Z)
    i, j = np.unravel_index(flat_idx, Z.shape)
    gm = (np.array([XX[i, j], YY[i, j]]), Z[i, j])
if local_maxs:
    gM = max(local_maxs, key=lambda t: t[1])
else:
    flat_idx = np.argmax(Z)
    i, j = np.unravel_index(flat_idx, Z.shape)
    gM = (np.array([XX[i, j], YY[i, j]]), Z[i, j])

return {"grid": (XX, YY, Z), "local_mins": local_mins, "local_maxs": local_maxs,
"global_min": gm, "global_max": gM, "bounds": bounds_pair}
res_r = search_extrema(rastrigin, [[dom_r[0], dom_r[1]], [dom_r[0], dom_r[1]]],
n_grid=201, n_starts=100, refine=True)
res_s = search_extrema(stenger, [[dom_s[0], dom_s[1]], [dom_s[0], dom_s[1]]],
n_grid=201, n_starts=100, refine=True)
def summary(name, res):
    print(f"\n{name}")
    b = res["bounds"]
    def fmt_point(pt):
        x, val = pt
        feas = in_bounds(x, b)
        return f"x* = ({x[0]:.6f}, {x[1]:.6f}), f = {val:.6f}, feasible={feas}"
    print("\nLocal min:")
    if len(res["local_mins"])==0:
        print("  None detected")
    else:
        for i, pt in enumerate(res["local_mins"]):
            print(f"  {i+1}. {fmt_point(pt)}")
    print("\nLocal max:")
    if len(res["local_maxs"])==0:
        print("  None detected")
    else:
        for i, pt in enumerate(res["local_maxs"]):
            print(f"  {i+1}. {fmt_point(pt)}")
    print("\nGlobal minimum:")
    print(" ", fmt_point(res["global_min"]))
    print("Global maximum:")
    print(" ", fmt_point(res["global_max"]))

summary("Rastrigin (c=5, p=10, f0=-100)", res_r)
summary("Stenger (a=-10, b=-1, c=+2)", res_s)
def plot_2d(name, res):
    XX, YY, Z = res["grid"]
    plt.figure(figsize=(8,6))
    cs = plt.contour(XX, YY, Z, levels=10)
    plt.clabel(cs, inline=True)
    if res["local_mins"]:

```

```

        xs, ys = zip(*[x for x, _ in res["local_mins"]])
        plt.scatter(xs, ys, c='blue', marker='o', label="local min")
    if res["local_maxs"]:
        xs, ys = zip(*[x for x, _ in res["local_maxs"]])
        plt.scatter(xs, ys, c='red', marker='s', label="local max")
    gx, gv = res["global_min"]
    plt.scatter(gx[0], gx[1], c='green', marker='^', s=200, label="global min")
    Gx, Gv = res["global_max"]
    plt.scatter(Gx[0], Gx[1], c='purple', marker='v', s=200, label="global max")
    plt.title(f"{name} - Contour")
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.legend(loc="upper right")
    plt.show()
def plot_3d(name, res, elev=30, azim=-60):
    XX, YY, Z = res["grid"]
    fig = plt.figure(figsize=(8,6))
    ax = fig.add_subplot(projection='3d')
    ax.plot_surface(XX, YY, Z)
    if res["local_mins"]:
        for x, val in res["local_mins"]:
            ax.scatter(x[0], x[1], val, c='blue', marker='o', s=40)
    if res["local_maxs"]:
        for x, val in res["local_maxs"]:
            ax.scatter(x[0], x[1], val, c='red', marker='s', s=40)
    gx, gv = res["global_min"]
    ax.scatter(gx[0], gx[1], gv, c='green', marker='^', s=200)
    Gx, Gv = res["global_max"]
    ax.scatter(Gx[0], Gx[1], Gv, c='purple', marker='v', s=200)
    ax.view_init(elev=elev, azim=azim)
    ax.set_title(f"{name} - Surface")
    ax.set_xlabel("x1")
    ax.set_ylabel("x2")
    ax.set_zlabel("f(x)")
    plt.show()
plot_2d("Rastrigin (c=5, p=10, f0=-100)", res_r)
plot_3d("Rastrigin (c=5, p=10, f0=-100)", res_r)
plot_2d("Stenger (a=-10, b=-1, c=+2)", res_s)
plot_3d("Stenger (a=-10, b=-1, c=+2)", res_s)
#Задание 3
def f31(x):
    return x[0]**2 + x[1]**2
def f32(x):
    return x[0]**2 + x[1]**2 - 0.5*x[0]*x[1]
cons_1 = [
    {'type':'ineq', 'fun': lambda x: 1 - (x[0] + x[1])},
    {'type':'ineq', 'fun': lambda x: x[0]},
    {'type':'ineq', 'fun': lambda x: x[1]},
]
cons_2 = [
    {'type':'ineq', 'fun': lambda x: 1 - (x[0]**2 + 2*x[1]**2)},
    {'type':'ineq', 'fun': lambda x: x[0]},
    {'type':'ineq', 'fun': lambda x: x[1]},
]
def draw_1(ax, dom):
    tri = np.array([[0,0],[1,0],[0,1],[0,0]])
    ax.plot(tri[:,0], tri[:,1], 'k')
    xs = np.linspace(0,1,200); ys = np.linspace(0,1,200)
    X,Y = np.meshgrid(xs,ys)
    mask = (X>=0)&(Y>=0)&(X+Y<=1)
    ax.scatter(X[mask], Y[mask], s=1, c='gray', alpha=0.3)
def draw_2(ax, dom):

```

```

t = np.linspace(0, np.pi/2, 400)
x = np.cos(t)
y = np.sin(t)/np.sqrt(2)
ax.plot(x, y, 'k')
xs = np.linspace(0,1,200)
ys = np.linspace(0,1/np.sqrt(2), 200)
X,Y = np.meshgrid(xs,ys)
mask = (X>=0)&(Y>=0)&(X**2 + 2*Y**2 <= 1 + 1e-12)
ax.scatter(X[mask], Y[mask], s=1, c='gray', alpha=0.3)
def minmax(f, cons, starts):
    def feasible(x):
        for c in (cons if isinstance(cons, (list,tuple))
                    else [cons]):
            v = c["fun"](x)
            if c["type"] == "eq":
                if np.any(np.abs(v) > 1e-6):
                    return False
            else:
                if np.any(v < -1e-6):
                    return False
        return True

    best_min = None
    best_max = None
    for x0 in starts:
        rmin = minimize(f, x0, method="SLSQP", constraints=cons,
                        options={"maxiter":1000,"ftol":1e-10})
        if rmin.success and feasible(rmin.x):
            val = f(rmin.x)
            if best_min is None or val < best_min[1]:
                best_min = (rmin.x, val)
        rmax = minimize(lambda u: -f(u), x0, method="SLSQP", constraints=cons,
                        options={"maxiter":1000,"ftol":1e-10})
        if rmax.success and feasible(rmax.x):
            val = f(rmax.x)
            if best_max is None or val > best_max[1]:
                best_max = (rmax.x, val)
    return best_min, best_max
def plot_3d(title, f, cons, draw_set, dom, nstarts=80, ngrid=100):
    x1 = np.linspace(dom[0], dom[1], ngrid)
    x2 = np.linspace(dom[2], dom[3], ngrid)
    X1, X2 = np.meshgrid(x1, x2)
    Z = np.vectorize(lambda a,b: f([a,b]))(X1, X2)
    rng = np.random.default_rng(0)
    starts = rng.uniform([dom[0], dom[2]], [dom[1], dom[3]], size=(nstarts,2))
    (xmin,fmin), (xmax,fmax) = minmax(f, cons, starts)

    plt.figure(figsize=(7,6))
    cs = plt.contour(X1, X2, Z, levels=15)
    plt.clabel(cs, inline=True, fontsize=9)
    plt.title(title)
    plt.xlabel("x1")
    plt.ylabel("x2")
    ax = plt.gca()
    draw_set(ax, dom)
    plt.scatter(xmin[0], xmin[1], c='green', marker='*', s=220, label="C.min")
    plt.scatter(xmax[0], xmax[1], c='magenta', marker='v', s=220, label="C.max")
    plt.legend(loc="upper right")
    plt.show()

    fig = plt.figure(figsize=(8,6))
    ax = fig.add_subplot(projection='3d')

```

```

ax.plot_surface(X1, X2, Z, cmap="viridis", alpha=0.8)
ax.scatter(xmin[0], xmin[1], fmin, c='green', marker='*', s=200, label="C.min")
ax.scatter(xmax[0], xmax[1], fmax, c='magenta', marker='v', s=200, label="C.max")
ax.set_title(title)
ax.set_xlabel("x1"); ax.set_ylabel("x2"); ax.set_zlabel("f(x)")
ax.legend(loc="best")
plt.show()

print("Условный минимум:", xmin, "f=", fmin)
print("Условный максимум:", xmax, "f=", fmax)
plot_3d("f=x1^2+x2^2", f31, cons_1, draw_1, dom=(-0.5,1.5,-0.5,1.5))
plot_3d("f=x1^2+x2^2 - 0.5*x1*x2", f32, cons_2, draw_2, dom=(-0.5,1.5,-0.5,1.0))

```